第 1 课 追小球的小车

【课程目标】：

本课来实现小车追小球的功能。

【课前准备】：

OpenMV、扩展板、云台、底座、zlink 模块、电源、总线双路驱动

【本课内容】：

本节来学习让小车追小球，具体控制小车运动使用的是 pid 算法

pid 算法是控制中运用非常多的一个算法，原理网上有很多，可以参考下面这两个链接：

https://zh.wikipedia.org/wiki/PID 控制器

http://baike.baidu.com/link?url=-obQq78Ur4bTeqA10bIniO6y0euQFcWL9WW18vq2hA3fyHN3rt32o79F2WPE7cK0Di9M6904rlHD9ttvVTySIK

一、代码

```
# 追小球的小车
import sensor, image, time
from pyb import LED
from ZL_SDK import zl_car_run, zl_pan_tilt, zl_uart3
from ZL_SDK.pid import PID

target_threshold = (15, 58, 39, 74, 62, 18)    # 颜色阈值
size_threshold = 475                           # 定义一个目标阈值区域的像素点数，小于这个阈值时使小车前进，大于这个阈值时使小车后退
x_pid = PID(p=0.7, i=1, imax=100)              # 控制小车方向
h_pid = PID(p=0.13, i=0.1, imax=50)            # 控制小车速度

# 初始化函数
def init_setup():
    global sensor, clock                       # 设置为全局变量
    sensor.reset()                             # 初始化感光元件
    sensor.set_vflip(True)
    sensor.set_hmirror(True)
```

```
    sensor.set_pixformat(sensor.RGB565)        # 将图像格式设置为彩色
    sensor.set_framesize(sensor.QQVGA)         # 将图像大小设置为 QQVGA，即 160*120
    sensor.skip_frames(10)                     # 跳过 10 帧使以上设置生效
    sensor.set_auto_whitebal(False)            # 关闭白平衡
    clock = time.clock()                       # 创建时钟对象
    zl_pan_tilt.main(0, -50)                   # 云台倾斜向下，方便查找目标


# 查找图像中最大的色块
def find_max(blobs):
    max_size = 0
    for blob in blobs:
        if blob[2] * blob[3] > max_size:
            max_blob = blob
            max_size = blob[2] * blob[3]
    return max_blob

# 主函数
def main():
    clock.tick()                                          # 更新 FPS 帧率时钟
    img = sensor.snapshot()                               # 拍一张照片并返回图像

    blobs = img.find_blobs([target_threshold])            # 在图像中查找目标颜色区域
    if blobs:                                             # 当查找到目标颜色区域后
        max_blob = find_max(blobs)                        # 查找图像中最大的目标色块
        x_error = max_blob[5] - img.width() / 2           # x 轴差值
        print(max_blob[2]*max_blob[3])
        h_error = max_blob[2] * max_blob[3] - size_threshold      # 面积差值
        print("x error: %s , h error: %s" % (x_error, h_error))
        img.draw_rectangle(max_blob[0:4])                     # 将找到的目标颜色区域用矩形框出来
        img.draw_cross(max_blob[5], max_blob[6])              # 在目标颜色区域的中心点处画十字
        x_output = x_pid.get_pid(x_error, 1)
        h_output = h_pid.get_pid(h_error, 1)
        print("x_output %s ,h_output %s" % (x_output, h_output))
        zl_car_run.car_run(int(-h_output-x_output)*10, int(-h_output+x_output)*10)  # 小车开始追小球
    else:
        zl_car_run.car_run(400, -400)                         # 当未找到目标颜色区域时，让小车原地转圈找寻目标颜色

# 程序入口
if __name__ == '__main__':
    init_setup()                # 执行初始化函数
```

```
zl_uart3.init_setup()        # 串口初始化
try:
    while 1:
        main()               # 执行主函数
except:
    zl_pan_tilt.middle()     # 云台复位
    zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)  # 小车停止
```

二、代码使用方法

1.根据实际情况修改 car_follow_color.py 代码中的如下四项：

```
target_threshold = (15, 58, 39, 74, 62, 18)    # 颜色阈值
size_threshold = 475                           # 定义一个目标阈值区域的像素点数，小于这个阈值时使小车前进，大于这个阈值时使小车
x_pid = PID(p=0.7, i=1, imax=100)              # 控制小车方向
h_pid = PID(p=0.13, i=0.1, imax=50)            # 控制小车速度
```

target_threshold 是小球的颜色阈值，可以根据实际情况来修改阈值，我这里使用的是一个红色木块。

size_threshold 是小球的像素点数，当小于这个像素点数时小车前进，大于这个像素点数时小车后退。可以根据实际情况修改该值。我这里使用的是 3cm*3cm 的红色小木块，对应 size_threshold 值大约为 475.

x_pid 控制的是小车的方向，也就是如果你发现小车方向拐的太大时，可以将该值调小一些来使用。

h_pid 控制的是小车的速度，也就是当你发现小车追小球时速度稍微快了一些，可以将该值调小一些来使用。

2.将 ZL_SDK 文件夹保存到 openmv 的盘符中，然后在 openmv ide 中把 car_follow_color.py 保存到 openmv 中脱机运行使用即可，注意运行之前调整颜色阈值（调整阈值可参考颜色识别章节）

三、运行效果

小车会跟随小球（注意不一定是小球，其他也可以，比如我这里使用一个红色小木块）移动，当小球靠近小车时，小车后退，当小球远离小车时，小车跟着前进，并且当小球拐弯时，小车也会跟着拐弯。当小球未出现在小车的视野内时，小车会在原地转圈寻找小球，找到小球后继续跟随。

四、知识点

1.本节控制小车运动使用的是 pid 算法

pid 算法是控制中运用非常多的一个算法，原理网上有很多，可以参考下面这两个链接：

https://zh.wikipedia.org/wiki/PID 控制器

http://baike.baidu.com/link?url=-obQq78Ur4bTeqA10bIniO6y0euQFcWL9WW18vq2hA3fyHN3rt32o79F2WPE7cK0Di9M6904rlHD9ttvVTySIK

2.在主代码中，具体需要修改的就是如下四项：

```
target_threshold = (15, 58, 39, 74, 62, 18)     # 颜色阈值
size_threshold = 475                            # 定义一个目标阈值区域的像素点数，小于这个阈值时使小车前进，大于这个阈值时使小车
x_pid = PID(p=0.7, i=1, imax=100)               # 控制小车方向
h_pid = PID(p=0.13, i=0.1, imax=50)             # 控制小车速度
```

target_threshold 是小球的颜色阈值，可以根据实际情况来修改阈值，我这里使用的是一个红色木块。

size_threshold 是小球的像素点数，当小于这个像素点数时小车前进，大于这个像素点数时小车后退。可以根据实际情况修改该值。我这里使用的是 3cm*3cm 的红色小木块，对应 size_threshold 值大约为 475.

x_pid 控制的是小车的方向，也就是如果你发现小车方向拐的太大时，可以将该值调小一些来使用。

h_pid 控制的是小车的速度，也就是当你发现小车追小球时速度稍微快了一些，可以将该值调小一些来使用。

3.关于小车追小球也可参考官方文档及教程：

文档：

https://book.openmv.cc/project/zhui-xiao-qiu-de-xiao-8f665d28-project-pan-tilt-md.html

视频教程：https://singtown.com/learn/49239/

【本课小结】：

本节学习了让小车追小球，具体控制小车运动使用的是 pid 算法

第 2 课 巡线小车

【课程目标】：

本课学习通过视觉识别、快速线性回归的方法来实现巡线功能

【课前准备】：

OpenMV、扩展板、云台、底座、zlink 模块、电源、总线双路驱动

【本课内容】：

使用快速线性回归来实现巡线

快速线性回归，可以使用 get_regression 快速返回视野中的一条回归直线，可以得到直线的角度 theta、直线的偏移距离 rho，然后根据角度和偏移距离动态的调整小车的速度。

一、代码

```
# 巡线小车
# 使用快速线性回归来实现
# 快速线性回归，可以使用 get_regression 快速返回视野中的一条回归直线，
# 可以得到直线的角度 theta、直线的偏移距离 rho，然后根据角度和偏移距离动态的调整小车的速度
import sensor, image, time
from pyb import LED
from ZL_SDK import zl_car_run, zl_pan_tilt, zl_uart3
from ZL_SDK.pid import PID

THRESHOLD = (32, 6, -23, 9, -18, 45)        # 线的颜色阈值，此处为白底黑线

rho_pid = PID(p=0.5, i=0)                   # 控制线在视野中的位置，即处于图像的左侧右侧或中间位置
theta_pid = PID(p=0.005, i=0)               # 控制直线角度的偏移

# 初始化函数
def init_setup():
    global sensor, clock                    # 设置为全局变量

    LED(1).on()                             # 打开板载 LED 灯补光，使巡线环境稳定一些
    LED(2).on()
    LED(3).on()

```

```
    sensor.reset()                          # 初始化感光元件
    sensor.set_pixformat(sensor.RGB565)     # 设置图像格式为彩色
        sensor.set_framesize(sensor.QQQVGA)         # 设置图像大小为 QQQVGA，80x60 (4,800 pixels) - O(N^2) max = 2,3040,000.
        sensor.skip_frames(10)                      # 跳过 10 帧使以上设置生效，WARNING: If you use QQVGA it may take seconds
    clock = time.clock()                    # 创建时钟对象
    zl_pan_tilt.main(0, -65)                # 云台倾斜向下，方便查找黑线

# 主函数
def main():
    clock.tick()                                    # 更新 FPS 帧率时钟
    img = sensor.snapshot().binary([THRESHOLD])     # 拍一张照片并返回图像，图像为二值化格式，即黑线部分显示为白色，非黑色区域显示为黑色
    line = img.get_regression([(100,100)], robust=True)     # 快速线性回归，可以快速返回视野中的一条回归直线，可得到直线的角度、偏移的距离
    if (line):
        rho_err = abs(line.rho()) - img.width() / 2         # 计算直线相对于图像中央偏移的距离
        if line.theta() > 90:                               # 通过对直线角度的判断来进行坐标的变换，即以直线最远点画竖线来作为 x 轴的零点位置
            theta_err = line.theta() - 180
        else:
            theta_err = line.theta()
        img.draw_line(line.line(), color=127)               # 将直线画出来
        print(rho_err,line.magnitude(),rho_err)
        if line.magnitude() > 8:                            # magnitude 为霍夫变换后的直线的模，值越大说明效果越好
            #if -40<b_err<40 and -30<t_err<30:
            rho_output = rho_pid.get_pid(rho_err, 1)
            theta_output = theta_pid.get_pid(theta_err, 1)
            output = rho_output + theta_output
            zl_car_run.car_run(int(50+output)*10, int(50-output)*10)  # 开始巡线
        else:
            zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)  # 小车停止
    else:
        zl_car_run.car_run(650,-650)  # 小车原地旋转，找寻黑线
        pass
    #print(clock.fps())

# 程序入口
if __name__ == '__main__':
    init_setup()    # 执行初始化函数
    zl_uart3.init_setup()
    try:
```

```
        while 1:
            main()  # 执行主函数
    except:
        zl_pan_tilt.middle()    # 云台复位
        zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)  # 小车停止
```

二、代码使用方法

将 ZL_SDK 文件夹保存到 openmv 的盘符中，然后在 openmv ide 中将 car_follow_line.py 存到 openmv 中脱机运行即可，注意运行之前可能需要根据环境调整颜色阈值

三、运行效果

将代码存到 openmv 之后，上电即可脱机运行，将小车放在黑线上，小车就可以巡黑线走了。

四、知识点

注意：rho_pid 控制线在视野中的位置，即处于图像的左侧右侧或中间位置

theta_pid 控制直线角度的偏移

```
rho_pid = PID(p=0.5, i=0)                # 控制线在视野中的位置，即处于图像的左侧右侧或中间位置
theta_pid = PID(p=0.005, i=0)            # 控制直线角度的偏移
```

关于 img.get_regression 方法可以在官方文档找到

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#

打开上述网址之后，按键盘快捷键 ctrl f，然后搜索 get_regression，可以查找到 image.get_regression 的使用方法及 Line 类 – 直线对象的使用。

关于巡线小车也可参考官方教程：https://singtown.com/learn/50037/

【本课小结】：

本课学习了通过视觉识别、快速线性回归的方法来实现巡线功能。

第 3 课 微信小程序控制

【课程目标】：

本课学习使用蓝牙模块通过微信小程序来控制 openmv 小车。

【课前准备】：

OpenMV、扩展总线板、云台、底座、zlink 模块、电源、总线双路驱动、蓝牙模块

【本课内容】：

本课学习使用蓝牙模块通过微信小程序来控制 openmv 小车，注意要将蓝牙模块插到 zlink 模块的 TTL 串口上使用。

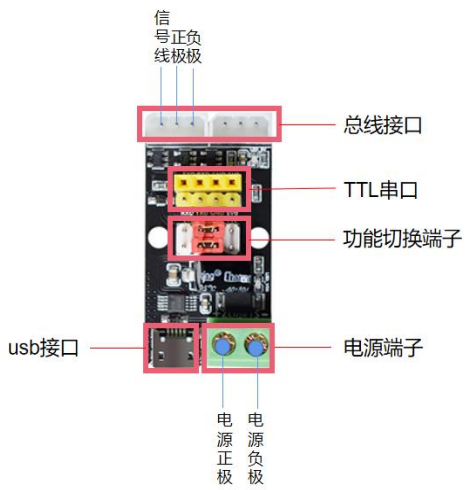
注意蓝牙模块的 VCC 对应 zlink 模块 TTL 串口的 5V0

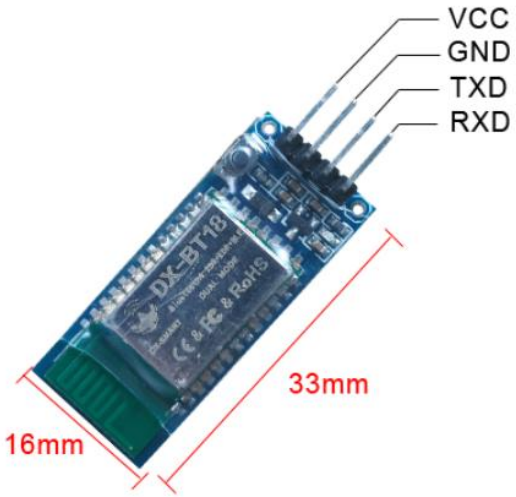
蓝牙模块的 GND 对应 zlink 模块 TTL 串口的 GND

蓝牙模块的 TXD 对应 zlink 模块 TTL 串口的 RXD

蓝牙模块的 RXD 对应 zlink 模块 TTL 串口的 TXD

功能切换端子的跳线帽要短接 TTL 和 ZX，即左侧





注意：本课代码同综合例程代码

本综合例程包含颜色识别、颜色跟随、人脸识别、人脸跟随、分辨人脸、二维码识别、数字识别、巡线小车、追小球的小车、前进、后退、左转、右转、停止、复位。

注意颜色相关的需要根据实际情况调节阈值范围并将代码中的阈值范围替换掉，分辨人脸需提前收集好人脸图片放在对应文件夹中并将代码按照前面的教程进行一些修改，具体使用方法在下面会看到。

一、代码

```
# 综合例程
# 包含颜色识别、颜色跟随、人脸识别、人脸跟随、分辨人脸、二维码识别、数字识别、巡线小车、追小球的小车
# 注意颜色相关的需要根据实际情况调节阈值范围，分辨人脸需提前收集好人脸图片
import sensor, image, time, pyb
from AI_Functions import (
    multi_color_detection, face_detection, color_tracking, face_tracking,
    face_recognition, qrcode_detection, mutil_template_num_matching,
    car_follow_color, car_follow_line
)
from ZL_SDK import (zl_uart3, zl_led, zl_car_run, zl_pan_tilt, pid)

# 串口接收数据后对应执行的命令值
uart_recv_data_command = None

# 1.对应每个功能的一个标志位，标志位用来初始化一次感光元件 2.串口接收到的对应数据 3.数据对应指令
flag_dict = {
    'multi_color_detection': {'flag': True, 'uart_recv_data': b'$YSSB!', 'command': 'YSSB'},
    'face_detection': {'flag': True, 'uart_recv_data': b'$RLSB!', 'command': 'RLSB'},
    'color_tracking': {'flag': True, 'uart_recv_data': b'$YSGS!', 'command': 'YSGS'},
    'face_tracking': {'flag': True, 'uart_recv_data': b'$RLGS!', 'command': 'RLGS'},
    'face_recognition': {'flag': True, 'uart_recv_data': b'$FBRL!', 'command': 'FBRL'},
    'qrcode_detection': {'flag': True, 'uart_recv_data': b'$EWMSB!', 'command': 'EWMSB'},
    'mutil_template_num_matching': {'flag': True, 'uart_recv_data': b'$SZSB!', 'command': 'SZSB'},
    'car_follow_color': {'flag': True, 'uart_recv_data': b'$ZXQ!', 'command': 'ZXQ'},
    'car_follow_line': {'flag': True, 'uart_recv_data': b'$XJMS!', 'command': 'XJMS'},
}
# 串口接收到对应指令控制小车运动及云台运动
motor_pan_tilt_command_dict = {
```

```
29.         '前进': b'$QJ!',
30.         '后退': b'$HT!',
31.         '左转': b'$ZZ!',
32.         '右转': b'$YZ!',
33.         '停止': b'$TZ!',
34.         '复位': b'$RESET!',
35.     }
36.
37.     # 标志位处理函数，标志位用来初始化一次感光元件，并让其他功能对应的 flag 变为真
38.     def flag_func(flag):
39.         for i in flag_dict:
40.             if i == flag:
41.                 flag_dict[i]['flag'] = False
42.             else:
43.                 flag_dict[i]['flag'] = True
44.         print(flag_dict)
45.
46.     # 串口接收数据的处理
47.     def uart_recv_data_handle():
48.         global uart_recv_data_command
49.         uart_recv_data = zl_uart3.main()
50.         #print(uart_recv_data, type(uart_recv_data))
51.         if uart_recv_data:
52.             if uart_recv_data == flag_dict['multi_color_detection']['uart_recv_data']:        # 颜色识别
53.                 uart_recv_data_command = flag_dict['multi_color_detection']['command']
54.             elif uart_recv_data == flag_dict['face_detection']['uart_recv_data']:             # 人脸识别
55.                 uart_recv_data_command = flag_dict['face_detection']['command']
56.             elif uart_recv_data == flag_dict['color_tracking']['uart_recv_data']:             # 颜色跟随
57.                 uart_recv_data_command = flag_dict['color_tracking']['command']
58.             elif uart_recv_data == flag_dict['face_tracking']['uart_recv_data']:              # 人脸跟随
59.                 uart_recv_data_command = flag_dict['face_tracking']['command']
60.             elif uart_recv_data == flag_dict['face_recognition']['uart_recv_data']:           # 分辨人脸
61.                 uart_recv_data_command = flag_dict['face_recognition']['command']
62.             elif uart_recv_data == flag_dict['qrcode_detection']['uart_recv_data']:           # 二维码识别
63.                 uart_recv_data_command = flag_dict['qrcode_detection']['command']
64.             elif uart_recv_data == flag_dict['mutil_template_num_matching']['uart_recv_data']: # 数字识别
65.                 uart_recv_data_command = flag_dict['mutil_template_num_matching']['command']
```

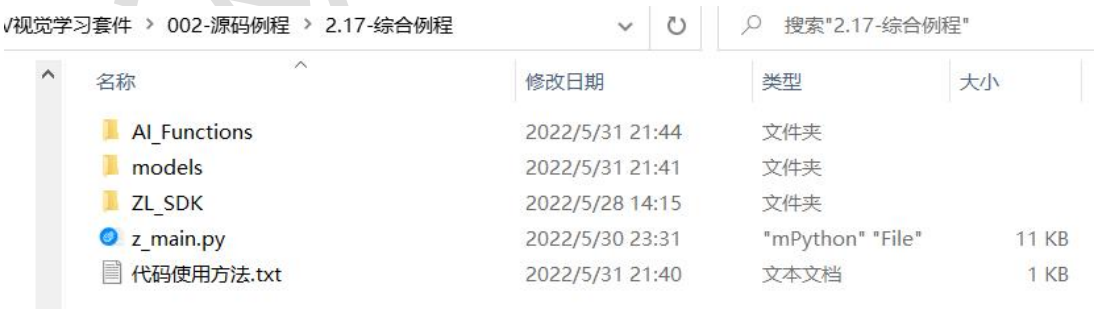
```
        elif uart_recv_data == flag_dict['car_follow_color']['uart_recv_data']:            # 追小球
            uart_recv_data_command = flag_dict['car_follow_color']['command']
        elif uart_recv_data == flag_dict['car_follow_line']['uart_recv_data']:            # 循迹模式
            uart_recv_data_command = flag_dict['car_follow_line']['command']
        elif uart_recv_data == motor_pan_tilt_command_dict['前进']:                        # 前进
            zl_car_run.car_run(zl_car_run.forward_speed, zl_car_run.forward_speed)
            uart_recv_data_command = 'QJ'
        elif uart_recv_data == motor_pan_tilt_command_dict['后退']:                        # 后退
            zl_car_run.car_run(zl_car_run.backward_speed, zl_car_run.backward_speed)
            uart_recv_data_command = 'HT'
        elif uart_recv_data == motor_pan_tilt_command_dict['左转']:                        # 左转
            zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.forward_speed)
            uart_recv_data_command = 'ZZ'
        elif uart_recv_data == motor_pan_tilt_command_dict['右转']:                        # 右转
            zl_car_run.car_run(zl_car_run.forward_speed, zl_car_run.stop_speed)
            uart_recv_data_command = 'YZ'
        elif uart_recv_data == motor_pan_tilt_command_dict['停止']:                        # 停止
            zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)
            uart_recv_data_command = 'TZ'
        elif uart_recv_data == motor_pan_tilt_command_dict['复位']:                        # 复位
            zl_pan_tilt.middle()
            uart_recv_data_command = 'FW'
        #print(uart_recv_data_command)


# 功能执行函数
def function_run():
    if uart_recv_data_command == flag_dict['multi_color_detection']['command']:  # 颜色识别功能
        if flag_dict['multi_color_detection']['flag']:                            # 只初始化一次
            multi_color_detection.init_setup()                                    # 颜色识别初始化
            flag_func('multi_color_detection')
        multi_color_detection.main()                                              # 执行颜色识别主函数

    elif uart_recv_data_command == flag_dict['face_detection']['command']:        # 人脸识别功能
        if flag_dict['face_detection']['flag']:                                   # 只初始化一次
            face_detection.init_setup()                                           # 人脸识别初始化
            flag_func('face_detection')
        face_detection.main()                                                     # 执行人脸识别主函数

    elif uart_recv_data_command == flag_dict['color_tracking']['command']:        # 颜色跟随功能
```

```
        if flag_dict['color_tracking']['flag']:                          # 只初始化一次
            color_tracking.init_setup()                                  # 颜色跟随初始化
            flag_func('color_tracking')
        color_tracking.main()                                            # 执行颜色跟随主函数

    elif uart_recv_data_command == flag_dict['face_tracking']['command']:        # 人脸跟随功能
        if flag_dict['face_tracking']['flag']:                           # 只初始化一次
            face_tracking.init_setup()                                   # 人脸跟随初始化
            flag_func('face_tracking')
        face_tracking.main()                                             # 执行人脸跟随主函数

    elif uart_recv_data_command == flag_dict['face_recognition']['command']:     # 分辨人脸功能
        if flag_dict['face_recognition']['flag']:                        # 只初始化一次
            face_recognition.init_setup()                                # 分辨人脸初始化
            flag_func('face_recognition')
        face_recognition.main()                                          # 执行分辨人脸主函数

    elif uart_recv_data_command == flag_dict['qrcode_detection']['command']:     # 二维码识别功能
        if flag_dict['qrcode_detection']['flag']:                        # 只初始化一次
            qrcode_detection.init_setup()                                # 二维码识别初始化
            flag_func('qrcode_detection')
        qrcode_detection.main()                                          # 执行二维码主函数

    elif uart_recv_data_command == flag_dict['mutil_template_num_matching']['command']:  # 数字识别功能
        if flag_dict['mutil_template_num_matching']['flag']:             # 只初始化一次
            mutil_template_num_matching.init_setup()                     # 数字识别初始化
            flag_func('mutil_template_num_matching')
        mutil_template_num_matching.main()                               # 执行数字识别主函数

    elif uart_recv_data_command == flag_dict['car_follow_color']['command']:     # 追小球功能
        if flag_dict['car_follow_color']['flag']:                        # 只初始化一次
            car_follow_color.init_setup()                                # 追小球功能初始化
            flag_func('car_follow_color')
        car_follow_color.main()                                          # 执行追小球主函数

```

```
    elif uart_recv_data_command == flag_dict['car_follow_line']['command']:      # 循迹功能
        if flag_dict['car_follow_line']['flag']:                                 # 只初始化一次
            car_follow_line.init_setup()                                         # 循迹模式初始化
            flag_func('car_follow_line')
        car_follow_line.main()                                                   # 执行循迹主函数

# 主函数
def main():
    uart_recv_data_handle()  # 串口接收数据并处理
    function_run()           # 根据接收到的数据执行不同的功能


# 程序入口
if __name__ == '__main__':
    zl_led.init_setup()      # led 灯初始化
    zl_uart3.init_setup()    # 串口初始化
    for i in range(6):       # led 灯亮三下，代表初始化完成，可以开始执行各项功能了
        zl_led.main(0.3)     # 每次亮 0.3 秒
    zl_pan_tilt.middle()     # 云台在程序开始后复位
    try:                     # 异常处理
        while 1:             # 无限循环
            main()           # 执行主函数
    except:
        zl_pan_tilt.middle() # 云台复位
        zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)  # 小车停止
```

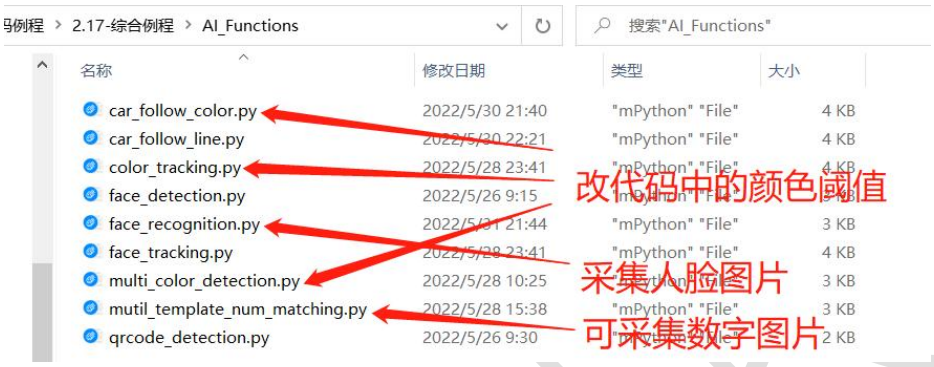
二、代码使用方法

1.在综合例程中共有如下几个文件和文件夹



2.在使用综合例程之前，需要将涉及到的颜色相关的代码，如颜色识别和颜色跟随代码中的颜色阈值按照前面的教程进行修改，分辨人脸的代码需要先采集人脸

图片并存放到 models 文件夹中的 ZLTech 中的对应位置，数字识别可以使用我们提供的模板，也可自己根据前面的教程采集数字图片模板并存放到 models 文件夹中的 template_num 中。



3.将 AI_Functions、models、ZL_SDK 文件夹都保存到 openmv 的盘符中，然后在 openmv ide 中将 z_main.py 文件存到 openmv 中，脱机运行即可，也可在线运行，不过要注意不要让小车跑了。

三、运行效果

代码运行后，我们主要通过蓝牙来控制小车。以下是各功能及对应指令：

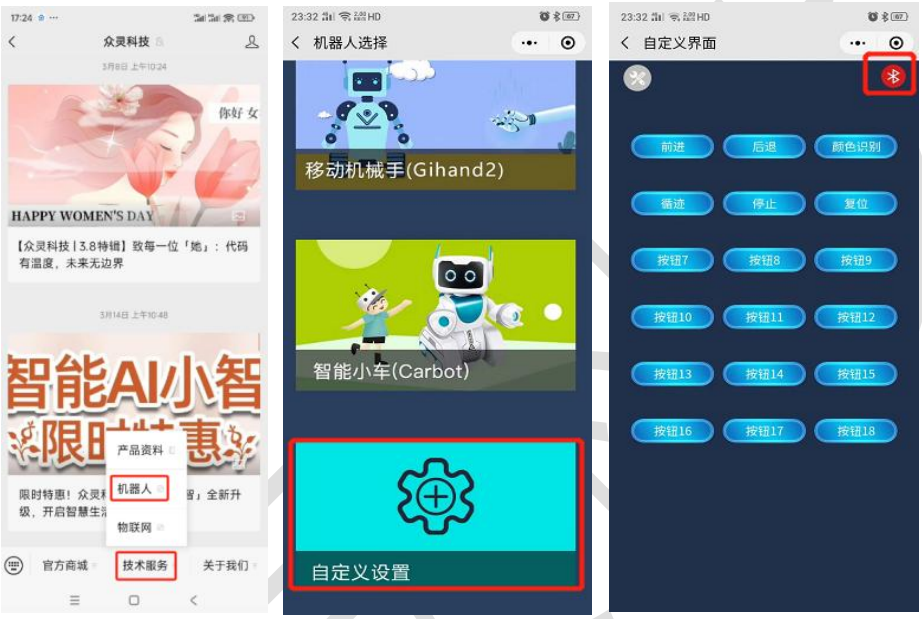
对应功能	指令
颜色识别	$YSSB!
颜色跟随	$YSGS!
人脸识别	$RLSB!
人脸跟随	$RLGS!
分辨人脸	$FBRL!
二维码识别	$EWMSB!
数字识别	$SZSB!
小车追小球	$ZXQ!
巡线小车	$XJMS!
小车前进	$QJ!
小车后退	$HT!
小车左转	$ZZ!
小车右转	$YZ!
小车停止	$TZ!
云台复位	$RESET!

蓝牙模块的基本用法：

1. 首先根据上面的教程将蓝牙模块插到 zlink 模块上，然后通过扩展板供电

2. 搜索微信公众号'众灵科技'，然后点击'**技术服务**'-'**机器人**'，点击 logo 进入小程序

3. 进入众灵机器人小程序，然后找到自定义设置，点击进入自定义设置，然后点击右上角的红色蓝牙图标



4.在弹出的界面中点击'点击连接'来连接蓝牙，注意如果你没有打开蓝牙的话，它会提醒你先打开蓝牙，连接成功后，会自动返回自定义设置界面，并且右上角的蓝牙图标会变为蓝色（下次如果连接的话会自动连接）。

5.点击左上角的设置图标，可以设置每个按钮的功能，注意设置完每个按钮之后都要点一下对应的'应用'按钮

比如这里我设置如下 2 个示例：

注意：此处我只设置了按下的命令，如果你需要按下和抬起都有对应的命令，也可以在对应功能的抬起命令那里填写对应指令。比如我这里可以将按钮 1 设置

为按下时前进，抬起时后退。

当我将所有功能都设置完成后，会如下图所示，当然你也可以变换位置：

然后就可以按这些配置好的按钮来操控小车了。

如下是各功能对应的指令：

对应功能	指令
颜色识别	$YSSB!
颜色跟随	$YSGS!
人脸识别	$RLSB!
人脸跟随	$RLGS!
分辨人脸	$FBRL!
二维码识别	$EWMSB!
数字识别	$SZSB!
小车追小球	$ZXQ!
巡线小车	$XJMS!
小车前进	$QJ!
小车后退	$HT!
小车左转	$ZZ!
小车右转	$YZ!
小车停止	$TZ!
云台复位	$RESET!

【本课小结】：

本课将本套件基本所有的功能做了一个整合，通过运行本程序来调用其他示例程序，然后通过小程序来调用。

注意在运行程序之前根据教程做好准备工作，不然程序可能出错。

众灵科技

第 4 课 语音交互

【课程目标】：

本课学习使用语音识别模块来控制 openmv 小车。

【课前准备】：

OpenMV、扩展板、云台、底座、zlink 模块、电源、总线双路驱动、语音识别模块

【本课内容】：

本综合例程包含颜色识别、颜色跟随、人脸识别、人脸跟随、分辨人脸、二维码识别、数字识别、巡线小车、追小球的小车、前进、后退、左转、右转、停止、复位。

注意颜色相关的需要根据实际情况调节阈值范围并将代码中的阈值范围替换掉，分辨人脸需提前收集好人脸图片放在对应文件夹中并将代码按照前面的教程进行一些修改，具体使用方法在下面会看到。

一、代码

```
# 综合例程
# 包含颜色识别、颜色跟随、人脸识别、人脸跟随、分辨人脸、二维码识别、数字识别、巡线小车、追小球的小车
# 注意颜色相关的需要根据实际情况调节阈值范围，分辨人脸需提前收集好人脸图片
import sensor, image, time, pyb
from AI_Functions import (
    multi_color_detection, face_detection, color_tracking, face_tracking,
    face_recognition, qrcode_detection, mutil_template_num_matching,
    car_follow_color, car_follow_line
)
from ZL_SDK import (zl_uart3, zl_led, zl_car_run, zl_pan_tilt, pid)

# 串口接收数据后对应执行的命令值
uart_recv_data_command = None

```

```
# 1.对应每个功能的一个标志位，标志位用来初始化一次感光元件 2.串口接收到的对应数据 3.数据对应指令
flag_dict = {
    'multi_color_detection': {'flag': True, 'uart_recv_data': b'$YSSB!', 'command': 'YSSB'},
    'face_detection': {'flag': True, 'uart_recv_data': b'$RLSB!', 'command': 'RLSB'},
    'color_tracking': {'flag': True, 'uart_recv_data': b'$YSGS!', 'command': 'YSGS'},
    'face_tracking': {'flag': True, 'uart_recv_data': b'$RLGS!', 'command': 'RLGS'},
    'face_recognition': {'flag': True, 'uart_recv_data': b'$FBRL!', 'command': 'FBRL'},
    'qrcode_detection': {'flag': True, 'uart_recv_data': b'$EWMSB!', 'command': 'EWMSB'},
    'mutil_template_num_matching': {'flag': True, 'uart_recv_data': b'$SZSB!', 'command': 'SZSB'},

    'car_follow_color': {'flag': True, 'uart_recv_data': b'$ZXQ!', 'command': 'ZXQ'},
    'car_follow_line': {'flag': True, 'uart_recv_data': b'$XJMS!', 'command': 'XJMS'},
}
# 串口接收到对应指令控制小车运动及云台运动
motor_pan_tilt_command_dict = {
    '前进': b'$QJ!',
    '后退': b'$HT!',
    '左转': b'$ZZ!',
    '右转': b'$YZ!',
    '停止': b'$TZ!',
    '复位': b'$RESET!',
}

# 标志位处理函数，标志位用来初始化一次感光元件，并让其他功能对应的 flag 变为真
def flag_func(flag):
    for i in flag_dict:
        if i == flag:
            flag_dict[i]['flag'] = False
        else:
            flag_dict[i]['flag'] = True
    print(flag_dict)

# 串口接收数据的处理
def uart_recv_data_handle():
    global uart_recv_data_command
    uart_recv_data = zl_uart3.main()
    #print(uart_recv_data, type(uart_recv_data))
    if uart_recv_data:
        if uart_recv_data == flag_dict['multi_color_detection']['uart_recv_data']:        # 颜色识别
            uart_recv_data_command = flag_dict['multi_color_detection']['command']
        elif uart_recv_data == flag_dict['face_detection']['uart_recv_data']:             # 人脸识别
            uart_recv_data_command = flag_dict['face_detection']['command']
```

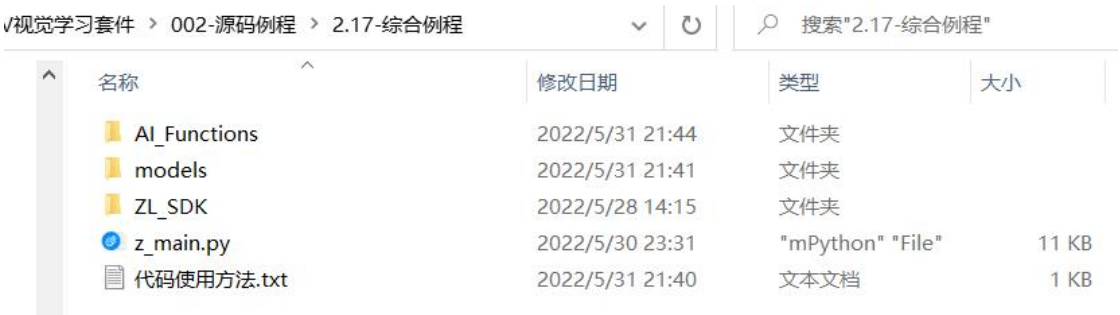
```
        elif uart_recv_data == flag_dict['color_tracking']['uart_recv_data']:            # 颜色跟随
            uart_recv_data_command = flag_dict['color_tracking']['command']
        elif uart_recv_data == flag_dict['face_tracking']['uart_recv_data']:             # 人脸跟随
            uart_recv_data_command = flag_dict['face_tracking']['command']
        elif uart_recv_data == flag_dict['face_recognition']['uart_recv_data']:          # 分辨人脸
            uart_recv_data_command = flag_dict['face_recognition']['command']
        elif uart_recv_data == flag_dict['qrcode_detection']['uart_recv_data']:          # 二维码识别
            uart_recv_data_command = flag_dict['qrcode_detection']['command']
        elif uart_recv_data == flag_dict['mutil_template_num_matching']['uart_recv_data']: # 数字识别
            uart_recv_data_command = flag_dict['mutil_template_num_matching']['command']
        elif uart_recv_data == flag_dict['car_follow_color']['uart_recv_data']:          # 追小球
            uart_recv_data_command = flag_dict['car_follow_color']['command']
        elif uart_recv_data == flag_dict['car_follow_line']['uart_recv_data']:           # 循迹模式
            uart_recv_data_command = flag_dict['car_follow_line']['command']
        elif uart_recv_data == motor_pan_tilt_command_dict['前进']:                      # 前进
            zl_car_run.car_run(zl_car_run.forward_speed, zl_car_run.forward_speed)
            uart_recv_data_command = 'QJ'
        elif uart_recv_data == motor_pan_tilt_command_dict['后退']:                      # 后退
            zl_car_run.car_run(zl_car_run.backward_speed, zl_car_run.backward_speed)
            uart_recv_data_command = 'HT'
        elif uart_recv_data == motor_pan_tilt_command_dict['左转']:                      # 左转
            zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.forward_speed)
            uart_recv_data_command = 'ZZ'
        elif uart_recv_data == motor_pan_tilt_command_dict['右转']:                      # 右转
            zl_car_run.car_run(zl_car_run.forward_speed, zl_car_run.stop_speed)
            uart_recv_data_command = 'YZ'
        elif uart_recv_data == motor_pan_tilt_command_dict['停止']:                      # 停止
            zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)
            uart_recv_data_command = 'TZ'
        elif uart_recv_data == motor_pan_tilt_command_dict['复位']:                      # 复位
            zl_pan_tilt.middle()
            uart_recv_data_command = 'FW'
        #print(uart_recv_data_command)


# 功能执行函数
def function_run():
```

```
    if uart_recv_data_command == flag_dict['multi_color_detection']['command']:  # 颜色识别功能
        if flag_dict['multi_color_detection']['flag']:                          # 只初始化一次
            multi_color_detection.init_setup()                                  # 颜色识别初始化
            flag_func('multi_color_detection')
        multi_color_detection.main()                                            # 执行颜色识别主函数

    elif uart_recv_data_command == flag_dict['face_detection']['command']:      # 人脸识别功能
        if flag_dict['face_detection']['flag']:                                 # 只初始化一次
            face_detection.init_setup()                                         # 人脸识别初始化
            flag_func('face_detection')
        face_detection.main()                                                   # 执行人脸识别主函数

    elif uart_recv_data_command == flag_dict['color_tracking']['command']:      # 颜色跟随功能
        if flag_dict['color_tracking']['flag']:                                 # 只初始化一次
            color_tracking.init_setup()                                         # 颜色跟随初始化
            flag_func('color_tracking')
        color_tracking.main()                                                   # 执行颜色跟随主函数

    elif uart_recv_data_command == flag_dict['face_tracking']['command']:       # 人脸跟随功能
        if flag_dict['face_tracking']['flag']:                                  # 只初始化一次
            face_tracking.init_setup()                                          # 人脸跟随初始化
            flag_func('face_tracking')
        face_tracking.main()                                                    # 执行人脸跟随主函数

    elif uart_recv_data_command == flag_dict['face_recognition']['command']:    # 分辨人脸功能
        if flag_dict['face_recognition']['flag']:                               # 只初始化一次
            face_recognition.init_setup()                                       # 分辨人脸初始化
            flag_func('face_recognition')
        face_recognition.main()                                                 # 执行分辨人脸主函数

    elif uart_recv_data_command == flag_dict['qrcode_detection']['command']:    # 二维码识别功能
        if flag_dict['qrcode_detection']['flag']:                               # 只初始化一次
            qrcode_detection.init_setup()                                       # 二维码识别初始化
            flag_func('qrcode_detection')
        qrcode_detection.main()                                                 # 执行二维码主函数

```

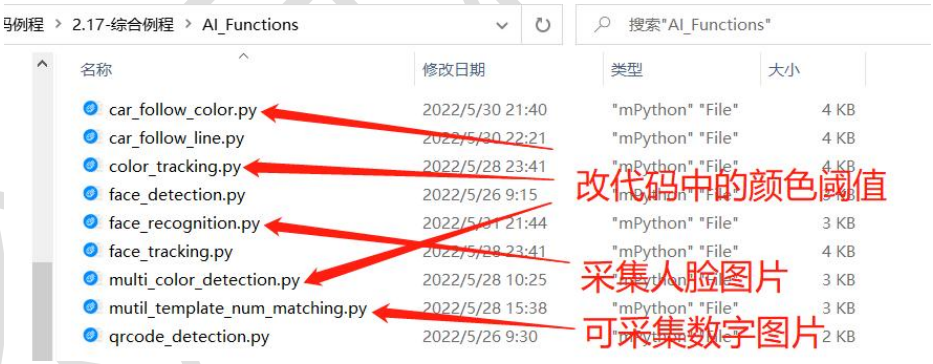
```
    elif uart_recv_data_command == flag_dict['mutil_template_num_matching']['command']:  # 数字识别功能
        if flag_dict['mutil_template_num_matching']['flag']:                    # 只初始化一次
            mutil_template_num_matching.init_setup()                            # 数字识别初始化
            flag_func('mutil_template_num_matching')
        mutil_template_num_matching.main()                                      # 执行数字识别主函数

    elif uart_recv_data_command == flag_dict['car_follow_color']['command']:    # 追小球功能
        if flag_dict['car_follow_color']['flag']:                               # 只初始化一次
            car_follow_color.init_setup()                                       # 追小球功能初始化
            flag_func('car_follow_color')
        car_follow_color.main()                                                 # 执行追小球主函数

    elif uart_recv_data_command == flag_dict['car_follow_line']['command']:     # 循迹功能
        if flag_dict['car_follow_line']['flag']:                                # 只初始化一次
            car_follow_line.init_setup()                                        # 循迹模式初始化
            flag_func('car_follow_line')
        car_follow_line.main()                                                  # 执行循迹主函数

# 主函数
def main():
    uart_recv_data_handle()  # 串口接收数据并处理
    function_run()           # 根据接收到的数据执行不同的功能


# 程序入口
if __name__ == '__main__':
    zl_led.init_setup()      # led 灯初始化
    zl_uart3.init_setup()    # 串口初始化
    for i in range(6):       # led 灯亮三下，代表初始化完成，可以开始执行各项功能了
        zl_led.main(0.3)     # 每次亮 0.3 秒
    zl_pan_tilt.middle()     # 云台在程序开始后复位
    try:                     # 异常处理
        while 1:             # 无限循环
            main()           # 执行主函数
    except:
        zl_pan_tilt.middle() # 云台复位
        zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)  # 小车停止
```

二、代码使用方法

1.在此例程中共有如下几个文件和文件夹



2.在使用综合例程之前，需要将涉及到的颜色相关的代码，如颜色识别和颜色跟随代码中的颜色阈值按照前面的教程进行修改，分辨人脸的代码需要先采集人脸图片并存放到 models 文件夹中的 ZLTech 中的对应位置，数字识别可以使用我们提供的模板，也可自己根据前面的教程采集数字图片模板并存放到 models 文件夹中的 template_num 中。



3.将 AI_Functions、models、ZL_SDK 文件夹都保存到 openmv 的盘符中，然后在 openmv ide 中将 z_main.py 文件存到 openmv 中，脱机运行即可，也可在线运行，不过要注意不要让小车跑了。

三、运行效果

代码运行后，我们主要通过语音来控制小车。

语音识别模块：

以下为对应的语音及功能：

颜色识别、颜色跟随、人脸识别、人脸跟随、分辨人脸、二维码识别、数字识别、追小球、循迹模式、前进、后退、左转、右转、停止、复位

语音模块的基本用法：你说'小灵小灵'或'你好小灵'可以将它唤醒，它会随机回复你，如果你将它唤醒后，在 20 秒内未用语音控制它，它会再次进入睡眠状态，等待你将它唤醒；

注意我们的语音功能都是需要在将它唤醒之后才控制它的。有些功能是可以使用不同的话语来控制的。你将它唤醒后，说'前进'或'往前走'，则小车会往前走；说'循迹模式'，则小车会开始循黑线。

唤醒词	小灵小灵
命令词	对应功能
颜色识别	颜色识别
颜色跟随	颜色跟随
人脸识别	人脸识别
人脸跟随	人脸跟随
分辨人脸	分辨人脸
二维码识别	二维码识别
数字识别	数字识别
追小球	小车追小球
循迹模式	巡线小车
前进、往前走	小车前进
后退、向后走、倒着走	小车后退
左转、向左转	小车左转
右转、向右转	小车右转
停止、停下	小车停止
复位	云台复位

【本课小结】：

本课将本套件基本所有的功能做了一个整合，通过运行本程序来调用其他示例程序，然后通过语音识别模块来控制。

注意在运行程序之前根据教程做好准备工作，不然程序可能出错。

众灵科技